

openagent-api

The Identity Gateway for OpenAgent — the FastAPI service that owns the OpenAgent persona, brokers every request between `openagent-frontend` and `openagent-infra`, and emits structured operational events to `openagent-logger`.

Overview

`openagent-api` is the product-layer backend in the OpenAgent system. It is the only HTTP boundary between `openagent-frontend` and `openagent-infra`, and it emits structured events to `openagent-logger`. It owns four concerns that nothing else in the system owns:

- The **OpenAgent persona** (`src/prompt/bio.txt`) — the system prompt that defines who the agent is.
- The **compartmentalized auth chain** — `OPENAGENT_API_KEY` validates the frontend inbound; `INFRA_API_KEY` authenticates openagent-api outbound to openagent-infra; `LOGGER_API_KEY` + `LOGGER_HMAC_SECRET` authenticate and sign openagent-api's outbound emissions to openagent-logger.
- The **SSE relay** — a transparent byte-for-byte pipe from openagent-infra's token stream to the frontend, with optional `reasoning_effort` pass-through.
- The **fire-and-forget logger emission** — every `/chat` call produces five structured events (`request_received`, `upstream_call`, [`upstream_error` if applicable], `stream_complete`, `conversation_capture`) that openagent-api enqueues for a background drain task to deliver to openagent-logger. The `/chat` hot path is never blocked on logger availability.

This repo is scoped to the gateway only. It has no model, no inference code, no database, no auth backend. All of those concerns live elsewhere.

The boundary is sharp by design: the frontend owns *how the agent looks*, openagent-api owns *who the agent is and who can talk to it*, openagent-infra owns *how models are accessed*, and openagent-logger owns *what was captured*. The four communicate over stable HTTP/SSE/HMAC contracts and never share code.

Where This Fits

```
openagent-os
├── openagent-infra      ← separate repo, separate compose stack
│   ├── Model proxy (port 8002 → BYOC Provider)
│   └── Forwards to base reasoning model and control layer model
├── openagent-api       ← YOU ARE HERE
│   └── The Identity Gateway (port 8001)
│       └── Owns the persona, the auth boundary,
```

the SSE relay between frontend and model,
and the fire-and-forget event emitter
to openagent-logger

- openagent-frontend ← separate repo, separate compose stack
 - └ The product UI (port 8000)
Streamlit chat interface, reaches openagent-api over HTTP
- openagent-logger ← separate repo, separate compose stack
 - └ The capture layer (port 8003)
Receives ops_events and conversation_captures from
openagent-api via fire-and-forget HMAC-signed HTTP;
stores in monthly-partitioned PostgreSQL

Port topology:

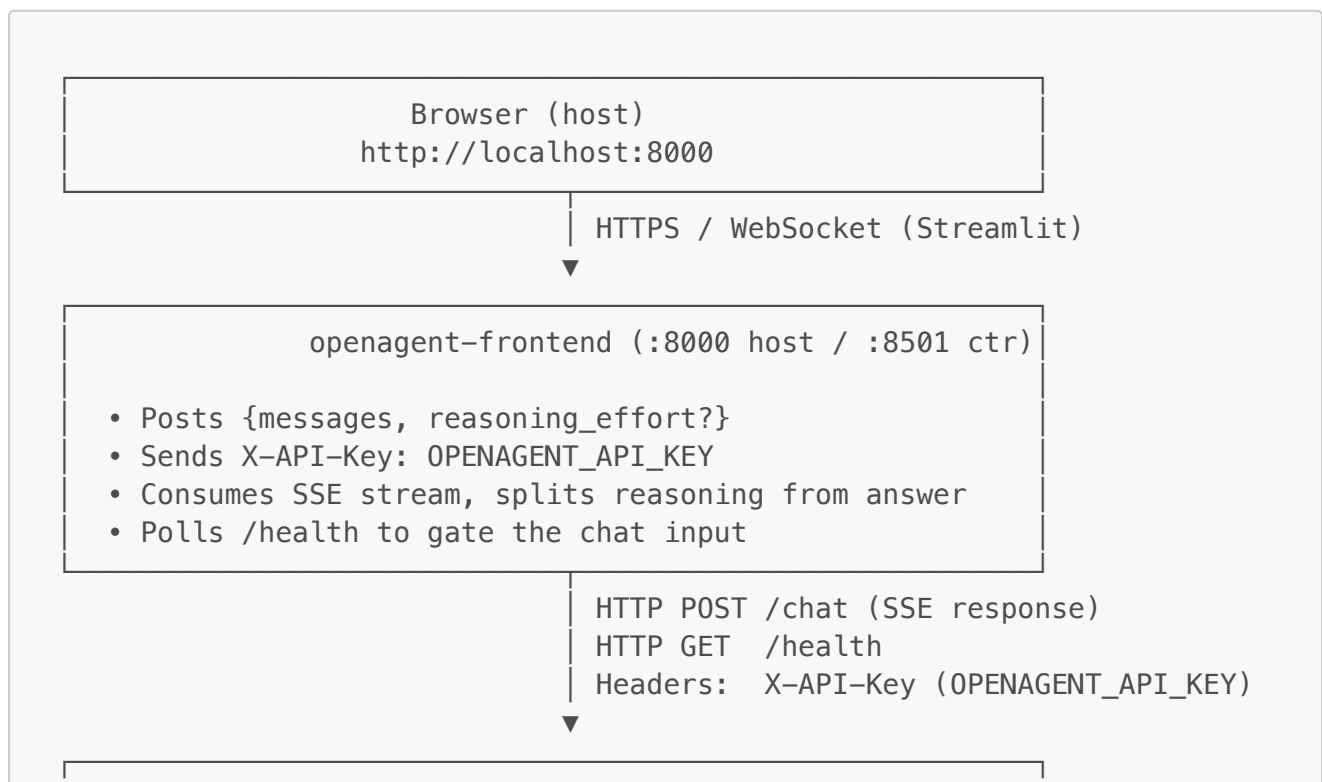
User → openagent-frontend (:8000) → openagent-api (:8001) → openagent-
infra (:8002) → BYOC Provider

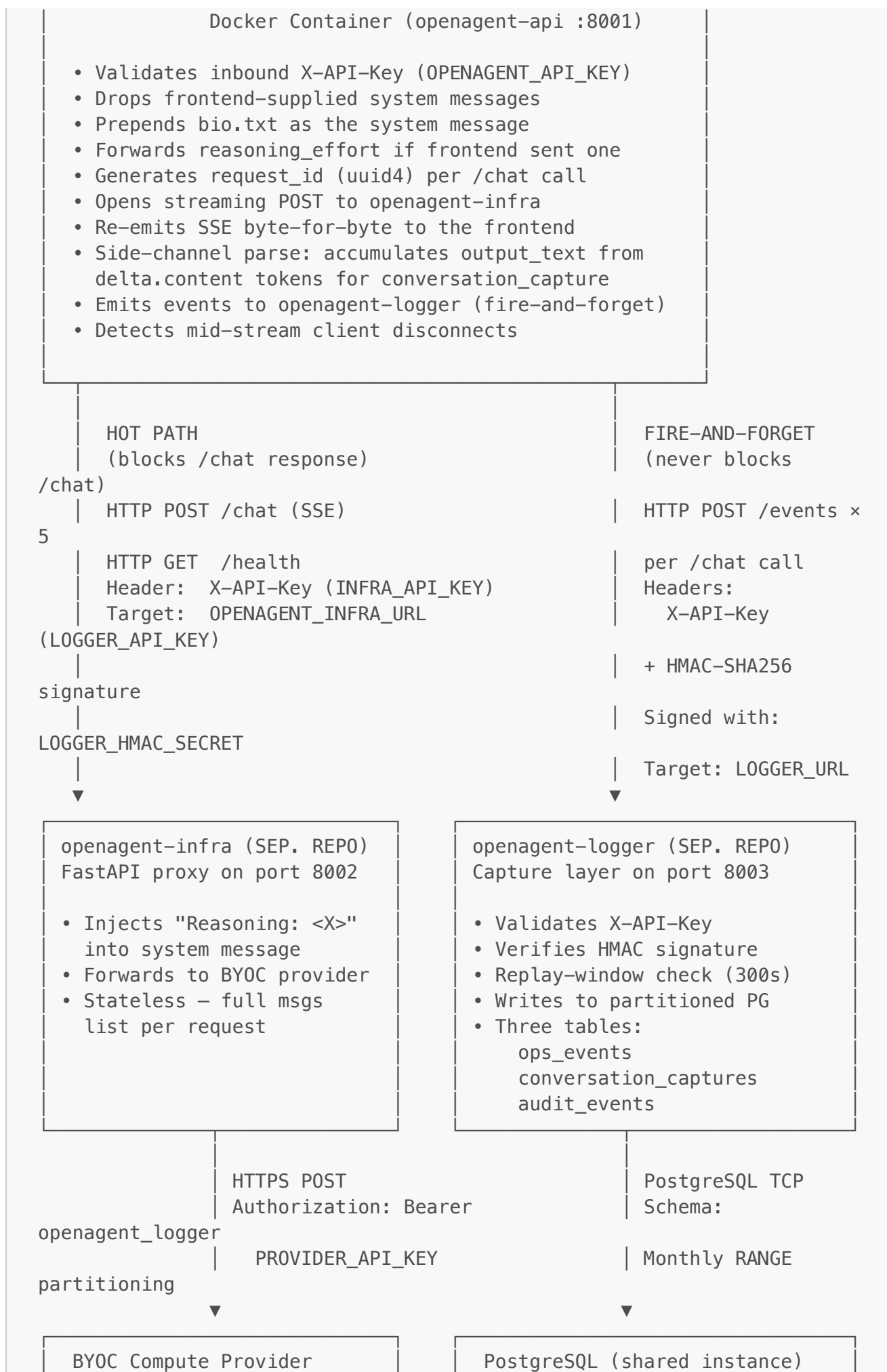
└ openagent-logger (:8003)

[fire-and-forget sibling]

Users only ever interact with port 8000. Port 8001 is internal to the product layer. Port 8002 is internal to the model layer and is reached only by openagent-api. Port 8003 is internal to the capture layer and is reached only by openagent-api. The external compute provider is reached only by openagent-infra.

Architecture





(e.g., RunPod, OpenAI) base reasoning model control layer model	
---	--

Request flow

1. The user sends a message in the openagent-frontend chat UI.
2. The frontend appends it to its in-session message list and POSTs `{messages: [...], reasoning_effort?: "..."} to openagent-api/chat with X-API-Key: OPENAGENT_API_KEY.`
3. `openagent-api` validates the inbound key (returns 401 if missing or wrong).
4. `openagent-api` generates a fresh `request_id` (UUID4) for the call. Every subsequent event emitted to openagent-logger for this `/chat` carries the same `request_id`, so the events can be joined post-hoc by querying openagent-logger's database.
5. `openagent-api` emits a `request_received` ops_event to openagent-logger (fire-and-forget, microseconds).
6. `openagent-api` filters out any `"system"` messages the frontend tried to send (defense in depth — the canonical persona lives here, not in the client).
7. `openagent-api` prepends `bio.txt` as the first `{"role": "system", "content": ...}` message and forwards the full payload (including `reasoning_effort` when the frontend sent one) to openagent-infra at `OPENAGENT_INFRA_URL/chat` with `X-API-Key: INFRA_API_KEY`. Immediately before the upstream POST, it emits an `upstream_call` ops_event.
8. openagent-infra injects `Reasoning: <level>` into the system message and forwards to the compute provider.
9. The provider streams tokens back through openagent-infra to openagent-api as SSE events.
10. `openagent-api` re-emits each event byte-for-byte to the frontend via `StreamingResponse`. The `data:` prefix and the `[DONE]` sentinel pass through unchanged. While the bytes pass through, a **side-channel parser** in `openagent-api` accumulates the `delta.content` tokens into an `output_text` buffer for the eventual conversation capture. The yield to the frontend happens BEFORE the parse in every iteration, so the parse adds zero latency to the user-visible stream.
11. When the upstream stream finishes cleanly, `openagent-api` emits a `stream_complete` ops_event (with `bytes_relayed` and `latency_ms` in details), then emits a `conversation_capture` carrying the assembled `output_text`, input/output hashes, model identifier, `reasoning_effort`, and latency. If the upstream fails at any point, `openagent-api` emits an `upstream_error` ops_event with the exception class name in details — no `stream_complete`, no `conversation_capture`.
12. If the frontend disconnects mid-stream, `openagent-api` detects it, abandons the upstream connection so the compute provider stops generating tokens nobody will read, and emits a `stream_complete` ops_event with `outcome=client_disconnect` (no `conversation_capture` for partial streams).

Separation of concerns

Concern	Lives In	Why
Persona / system prompt	openagent-api	Identity is product-layer backend logic; not UI logic
Inbound auth (frontend → api)	openagent-api	Auth boundaries belong on the server, not the client
Outbound auth (api → infra)	openagent-api	Upstream secret never leaves the backend network
Outbound auth (api → logger)	openagent-api	Logger secret never leaves the backend network
Request correlation ID	openagent-api	request_id (UUID4) generated per /chat, threads through every event emitted to openagent-logger
Event emission (fire-and-forget)	openagent-api	The /chat hot path must not block on the capture layer
Event capture & storage	openagent-logger	Sibling service; partitioned PostgreSQL, append-only
Conversation history	openagent-frontend	Held in-session by the frontend
Reasoning-format display	openagent-frontend	UX policy decision, not an inference concern
Reasoning effort default	openagent-infra	Server-side default lives upstream; openagent-api passes through only
Reasoning effort injection	openagent-infra	openagent-infra writes "Reasoning: " into system msg
Model serving & inference	BYOC Provider	Heavy, GPU-dependent

Compartmentalized auth chain

The auth model is **defense in depth** across multiple boundaries: independently generated secrets, validated at each boundary, with no key relayed unchanged across services.

```
frontend —[OPENAGENT_API_KEY]—> openagent-api —[INFRA_API_KEY]—>
openagent-infra
```

```
|
```

```
[PROVIDER_API_KEY]
```

```
▼
```

```
BYOC
```

Provider

openagent-logger

[LOGGER_API_KEY —>
+ LOGGER_HMAC_SECRET]

- **OPENAGENT_API_KEY** lives in **openagent-frontend/.env** and **openagent-api/.env**. openagent-api validates it on every inbound request.
- **INFRA_API_KEY** lives in **openagent-api/.env** and **openagent-infra/.env**. openagent-infra validates it on every inbound request.
- **LOGGER_API_KEY** lives in **openagent-api/.env** and **openagent-logger/.env**. openagent-logger validates it on every inbound **/events** request.
- **LOGGER_HMAC_SECRET** lives in **openagent-api/.env** and **openagent-logger/.env**. openagent-api signs every event payload with it; openagent-logger re-derives and verifies the signature on every received event AND stores the signature on the row for offline re-verification.
- **PROVIDER_API_KEY** lives in **openagent-infra/.env** only. openagent-api never sees it.

Each value is **independent**. The frontend never sees **INFRA_API_KEY**, **LOGGER_API_KEY**, **LOGGER_HMAC_SECRET**, or **PROVIDER_API_KEY**. The keys are not relayed.

Tech Stack

Layer	Technology
Base image	python:3.11-slim
API framework	FastAPI + uvicorn
HTTP client	httpx (async, with streaming support)
Env loading	python-dotenv
Validation	Pydantic (bundled with FastAPI)
HMAC signing	hmac + hashlib (Python stdlib)
Event queue	asyncio.Queue (Python stdlib)
Containerization	Docker + Docker Compose
Port (internal)	8001
Port (host)	8001
Auth (inbound)	API key via X-API-Key header
Auth (outbound to infra)	API key via X-API-Key header
Auth (outbound to logger)	API key via X-API-Key header + HMAC-SHA256 payload signature
Communication in	HTTP/1.1 (frontend → openagent-api)

Layer	Technology
Communication out	HTTP/1.1 + SSE (openagent-api → openagent-infra)
Communication out	HTTP/1.1 (openagent-api → openagent-logger, fire-and-forget JSON POST)

Intentionally absent: `groq`, `openai`, `torch`, `transformers`, `sqlalchemy`, `streamlit`, `cryptography`, `pynacl`, `arq`, `redis` — none of them belong in a stateless gateway. HMAC signing uses Python stdlib; the event queue is an `asyncio.Queue` from stdlib; both compose naturally with the existing httpx + FastAPI stack.

Prerequisites

- **Docker Desktop** (macOS / Windows) or **Docker Engine + Compose v2** (Linux)
- **openagent-infra running and reachable** — either on the host, in another Docker container, or deployed elsewhere. openagent-infra in turn requires a BYOC provider (e.g., RunPod, OpenAI).
- **openagent-logger running and reachable** — either on the host, in another Docker container, or deployed elsewhere. openagent-logger in turn requires a PostgreSQL 13+ instance.
- **Four valid secrets** — `OPENAGENT_API_KEY`, `INFRA_API_KEY`, `LOGGER_API_KEY`, `LOGGER_HMAC_SECRET`.

You do **not** need:

- A GPU (the gateway does no inference)
- Python installed on the host (Docker handles it)
- A compute provider API key (those belong to openagent-infra)
- Direct PostgreSQL access (that's openagent-logger's concern)

Project Structure

```

openagent-api/
├── docker/
│   └── api/
│       └── Dockerfile                # Python 3.11 slim, non-root,
healthcheck'd
├── src/
│   ├── backend/
│   │   └── api.py                  # The FastAPI app (single file)
│   ├── client/
│   │   ├── __init__.py            # Package marker
│   │   └── infra.py               # InfraClient – proxy to openagent-
infra
│   └── logger.py                  # LoggerClient – fire-and-forget to
openagent-logger
│   └── prompt/
│       └── bio.txt                 # OpenAgent system prompt (persona)

```

```
|— docker-compose.yml      # Single-service compose
|— requirements.txt        # fastapi, uvicorn, httpx, python-
dotenv
|— .env                    # secrets – never commit
|— .env.example            # template for .env
|— .dockerignore           # keeps secrets / caches out of
build context
|— .gitignore              # keeps secrets / caches out of git
|— README.md               # this file
```

Setup

1. Clone the repo

```
git clone https://github.com/william-mckee/openagent-api.git
cd openagent-api
```

2. Create your `.env` file

```
cp .env.example .env
```

Open `.env` and set the **six required values**:

```
OPENAGENT_API_KEY=<long random hex>
INFRA_API_KEY=<long random hex, MUST match openagent-infra's API_KEY>
OPENAGENT_INFRA_URL=http://host.docker.internal:8002

LOGGER_URL=http://host.docker.internal:8003
LOGGER_API_KEY=<copy from openagent-logger/.env LOGGER_API_KEY>
LOGGER_HMAC_SECRET=<copy from openagent-logger/.env LOGGER_HMAC_SECRET>
```

Generate fresh keys:

```
python -c "import secrets; print(secrets.token_hex(32))"
```

The keys **MUST** be different values. Reusing one defeats the entire compartmentalized auth design.

3. Update `bio.txt` with your persona

The persona lives in `src/prompt/bio.txt`. The repo ships with a generic starter persona; replace it with your own before running anything you care about. The file is read once at container startup and

prepended as the first `system` message on every `/chat` call.

The file is baked into the Docker image at build time. **Rebuilding is required to pick up changes** — or mount a volume over `/app/src/prompt` in `docker-compose.yml` for live edits during development.

4. Make sure `openagent-infra` and `openagent-logger` are running

`openagent-api` is a thin gateway. Without `openagent-infra` reachable at `OPENAGENT_INFRA_URL`, `/health` will report `unreachable`.

`openagent-api` refuses to boot if `LOGGER_URL`, `LOGGER_API_KEY`, or `LOGGER_HMAC_SECRET` is missing. It does NOT refuse to boot if `openagent-logger` is unreachable at startup — fire-and-forget is graceful about transient logger unavailability.

5. Build and start

```
docker-compose up -d --build
```

6. Verify

```
# Confirm /health is reachable and authenticated.
curl -H "X-API-Key: <your OPENAGENT_API_KEY>"
http://localhost:8001/health
```

To verify the logger integration is working end-to-end, send a real `/chat` call and then query `openagent-logger`:

```
# Send a /chat:
curl -N -X POST \
  -H "X-API-Key: <OPENAGENT_API_KEY>" \
  -H "Content-Type: application/json" \
  -d '{"messages":[{"role":"user","content":"say hi"}], "reasoning_effort":"low"}' \
  http://localhost:8001/chat

# Then check openagent-logger received the events:
curl -fsS -H "X-API-Key: <LOGGER_API_KEY>" http://localhost:8003/stats
```

How It Works

System prompt ownership

The persona is owned by this repo and nowhere else. The frontend does not carry it. `openagent-infra` has no knowledge of it.

Concretely:

1. At container startup, it reads `/app/src/prompt/bio.txt` once into memory.
2. On every `/chat` call, that content is prepended as `{"role": "system", "content": "<bio>"}` before the request is forwarded upstream.
3. If a frontend tries to send its own system message in the request body, it is dropped with a warning log.

Inbound auth (frontend → openagent-api)

Every `/chat` and `/health` request must include `X-API-Key` matching `OPENAGENT_API_KEY`. The validation is implemented as a FastAPI dependency.

Outbound auth (openagent-api → openagent-infra)

The outbound boundary is owned by the `InfraClient` class in `src/client/infra.py`. The client's internal `httpx.AsyncClient` is constructed once at `start()` time with `headers={"X-API-Key": INFRA_API_KEY}`. Every outbound call inherits this header automatically.

Outbound auth (openagent-api → openagent-logger)

The `LoggerClient` constructs its own `httpx.AsyncClient` at startup with `headers={"X-API-Key": LOGGER_API_KEY}` and `base_url=LOGGER_URL`. Every event POST inherits the header automatically.

In addition to the transport key, every event body carries an `hmac_signature` field — the HMAC-SHA256 of the canonical string `{request_id}|{client_timestamp_iso}|{event_type}|sha256(canonical_payload_json)`, keyed with `LOGGER_HMAC_SECRET`.

Reasoning effort pass-through

`reasoning_effort` is an optional field on the `/chat` request body. Three accepted values: `low`, `medium`, `high`. The behaviour is **pure pass-through**:

- When the frontend sends a value, openagent-api includes it in the upstream payload to openagent-infra.
- When the frontend omits the field, openagent-api also omits it from the upstream payload — openagent-infra applies its own server-side default.

SSE relay

The `/chat` endpoint is a `StreamingResponse` driven by an async generator (`sse_pump`). The generator opens a streaming POST to openagent-infra, loops on `response.aiter_raw()`, and yields each chunk straight to the frontend.

Health gate (proxied)

The frontend's `/health` polling loop talks to openagent-api. openagent-api forwards the question to openagent-infra and returns a status the frontend already understands.

Statelessness

openagent-api is **stateless across requests**. It holds no conversation memory, no session store, no per-user state. The frontend sends the full message list on every turn and the gateway forwards it without adding or remembering anything.

Security Model

The system uses a **compartmentalization** architecture for service-to-service authentication. Each service holds only the secrets for the boundaries it directly touches; nothing is "passed through" the chain unchanged. The pattern makes single-service compromise containable rather than catastrophic.

Blast-radius analysis

- If **openagent-frontend** is compromised: the attacker gets **OPENAGENT_API_KEY** only. They cannot reach openagent-infra, openagent-logger, or the compute provider directly.
- If **openagent-api** is compromised: the attacker gets **OPENAGENT_API_KEY**, **INFRA_API_KEY**, **LOGGER_API_KEY**, and **LOGGER_HMAC_SECRET**. They still don't have **PROVIDER_API_KEY** — they cannot bypass openagent-infra to bill the compute provider directly.
- If **openagent-infra** is compromised: the attacker gets **INFRA_API_KEY** and **PROVIDER_API_KEY**. The model layer is exposed, but openagent-api's inbound boundary stays safe, openagent-logger stays safe, and the frontend stays safe.
- If **openagent-logger** is compromised: the attacker gets **LOGGER_API_KEY** and **LOGGER_HMAC_SECRET**. They can read whatever events have been captured, but they cannot reach openagent-api's inbound, openagent-infra, or the compute provider.

Service-to-service vs user-to-service auth

The keys above all authenticate **services to services**, not users to services. They identify which service is talking, not which person. There is no concept of a user identity in the current system — **OPENAGENT_API_KEY** is a single shared secret between openagent-frontend and openagent-api.

Key rotation procedure

Rotating **LOGGER_API_KEY (transport)**: Rotation is **cheap** — the key only protects future submissions; existing rows in openagent-logger's database are unaffected.

Rotating **LOGGER_HMAC_SECRET (payload signing)**: Rotation is **expensive** — pre-rotation signatures cannot be re-verified with the new secret. Rotate only on known compromise.

Integration with openagent-logger

The integration is **fire-and-forget by design**: openagent-api emits events into an in-process queue and returns immediately; a background asyncio task drains the queue and POSTs to openagent-logger. The **/chat** hot path is never coupled to openagent-logger availability. If openagent-logger is unreachable, events queue up while **/chat** continues to serve normally.

What gets emitted

Five events are emitted per successful `/chat` call:

1. `request_received`
2. `upstream_call`
3. `upstream_error` (if applicable)
4. `stream_complete`
5. `conversation_capture` (after a successful `stream_complete` only)

Queue overflow: drop-oldest

The queue is bounded. When it fills (sustained openagent-logger outage, or massive burst of traffic), the policy is **drop-oldest with WARNING log**.

Required configuration

Three env vars are required at startup: `LOGGER_URL`, `LOGGER_API_KEY`, `LOGGER_HMAC_SECRET`.

API Reference

POST `/chat`

The main endpoint. Forward a list of user/assistant turns and receive a streamed response via Server-Sent Events.

Request headers:

```
Content-Type: application/json
X-API-Key: <OPENAGENT_API_KEY>
```

Request body:

```
{
  "messages": [
    {"role": "user",      "content": "What is the Fibonacci sequence?"},
    {"role": "assistant", "content": "The Fibonacci sequence is..."},
    {"role": "user",      "content": "Show me in Python"}
  ],
  "reasoning_effort": "medium"
}
```

Response: `text/event-stream`

GET `/health`

Proxied health check.

Request headers:

```
X-API-Key: <OPENAGENT_API_KEY>
```

GET /docs

Auto-generated Swagger UI. Open in a browser:

```
http://localhost:8001/docs
```

Configuration

All configuration is loaded from `.env` at the repository root via `python-dotenv` and `docker-compose`'s `env_file:` directive. See `.env.example` for the full template.

Required

Variable	Type	Description
<code>OPENAGENT_API_KEY</code>	string	Inbound auth secret. Validated on every <code>/chat</code> and <code>/health</code> request.
<code>INFRA_API_KEY</code>	string	Outbound auth secret to openagent-infra. Sent as X-API-Key on every call.
<code>OPENAGENT_INFRA_URL</code>	string	Base URL of openagent-infra. No trailing slash.
<code>LOGGER_URL</code>	string	Base URL of openagent-logger. No trailing slash.
<code>LOGGER_API_KEY</code>	string	Outbound transport secret to openagent-logger.
<code>LOGGER_HMAC_SECRET</code>	string	Payload-signing secret for events.

Optional

Variable	Default	Description
<code>OPENAGENT_FRONTEND_URL</code>	—	Extra CORS origin for production deployments.
<code>OPENAGENT_LOG_LEVEL</code>	<code>INFO</code>	DEBUG INFO WARNING ERROR CRITICAL

Variable	Default	Description
OPENAGENT_UPSTREAM_CONNECT_TIMEOUT	10.0	Seconds to wait for TCP connect to openagent-infra.
OPENAGENT_UPSTREAM_READ_TIMEOUT	none	Seconds (or none for unbounded) for streaming reads.
OPENAGENT_HEALTH_TIMEOUT	5.0	Seconds for upstream /health probe.
OPENAGENT_BIO_PATH	/app/src/prompt/bio.txt	Override path to bio.txt inside the container.
OPENAGENT_LOGGER_QUEUE_MAX_SIZE	1000	Max pending events in LoggerClient queue before drop-oldest.

Note: there is intentionally **no** **OPENAGENT_DEFAULT_REASONING_EFFORT**. openagent-api is pure pass-through for **reasoning_effort**.

Local Development (without Docker)

For faster iteration than a Docker rebuild, run uvicorn directly on the host:

```
# 1. Create a virtualenv
python3.11 -m venv .venv
source .venv/bin/activate    # Linux/macOS
# .venv\Scripts\activate    # Windows

# 2. Install dependencies
pip install -r requirements.txt

# 3. Export env or rely on .env (python-dotenv picks it up)
export OPENAGENT_API_KEY=...
export INFRA_API_KEY=...
export OPENAGENT_INFRA_URL=http://localhost:8002
export LOGGER_URL=http://localhost:8003
export LOGGER_API_KEY=...    # copy from openagent-logger/.env
export LOGGER_HMAC_SECRET=... # copy from openagent-logger/.env
export OPENAGENT_BIO_PATH=$(pwd)/src/prompt/bio.txt
```

```
# 4. Run uvicorn on port 8001
PYTHONPATH=src uvicorn backend.api:app --host 0.0.0.0 --port 8001 --
reload
```

Design Decisions

Why does the persona live here and not in the frontend?

Identity is product-layer **backend** logic, not UI logic. The frontend is replaceable; the identity is not.

Why two API keys instead of one shared secret?

Defense in depth. With one shared key, compromise of any of the four services compromises all four.

Why two secrets for openagent-logger (LOGGER_API_KEY + LOGGER_HMAC_SECRET)?

Different threat models and different rotation profiles. **LOGGER_API_KEY** (transport) gates wire-level access. **LOGGER_HMAC_SECRET** (payload signing) provides payload integrity for downstream consumers to verify event integrity offline.

Why fire-and-forget logger emission?

The /chat hot path must not block on the capture layer.

Why drop-oldest queue overflow?

When the queue fills (sustained openagent-logger outage), fresher data is more useful than stale data when the storm passes.

Why no reasoning chain capture?

conversation_capture.output_text contains only **delta.content** tokens (the visible answer), not **delta.reasoning** tokens (the chain-of-thought). Capturing the chain couples the capture schema to a specific model's emission format.

Why is **reasoning_effort** pass-through with no api default?

The setting has one source of truth: openagent-infra's env var.

Why SSE relay instead of buffering and returning JSON?

Buffering means a multi-minute spinner with no user feedback. Streaming gives them tokens as they arrive — the only sane UX for this latency profile.

Why no conversation history at this layer?

Statelessness. openagent-api is deliberately a pure gateway with no per-session memory.

Why httpx instead of requests or aiohttp?

`httpx` is async-native (FastAPI's event loop can `await` it), supports streaming exactly as the SSE pump needs, and has a familiar API.

Why an in-memory queue instead of a durable outbox?

Simplicity vs. durability trade-off. An **in-memory `asyncio.Queue`** is zero infrastructure: no new dependencies, no new services, no new failure modes. The cost is that events queued in memory at the moment of a container restart are lost.

Why is the `/health` endpoint authenticated?

It reports operational state. That's internal information.

Why a non-root user in the container?

A container escape that gives root on the container should not give root on the host. The user (uid 1000) is created in the Dockerfile and the runtime drops to it before uvicorn starts.

Why an unbounded read timeout?

A finite read timeout would kill long but legitimate generations. Connect timeout is short (10s) because if openagent-infra itself is unreachable, we want to fail fast.

Why port 8001?

8000 = openagent-frontend (user-facing), 8001 = openagent-api (the API the frontend talks to), 8002 = openagent-infra (model layer, internal), 8003 = openagent-logger (capture layer, internal). The numbering reflects the request flow.

Known Limitations

These are present-tense limitations I'm aware of and accept for now.

No persistent conversation history

History is held client-side by the frontend. Closing the browser tab loses everything; the gateway itself remembers nothing across requests.

No per-user authentication

`OPENAGENT_API_KEY` is a shared secret between the frontend and the gateway, not a per-user credential. There is no "user A vs user B" at this layer. Not appropriate for exposure beyond a trusted operator without an auth layer in front.

No rate limiting or abuse protection

The gateway trusts authenticated clients. Safe because the frontend is the only client. If exposed more widely, rate limiting would belong at a reverse proxy in front of the gateway.

Event capture is best-effort

The capture pipeline is fire-and-forget with an in-memory queue, so events can be lost if openagent-logger is unreachable when an event is emitted, or if the queue fills during a sustained outage (drop-oldest). `/chat` is unaffected by this — losing capture events never degrades the user-facing response.

Context-window handling is upstream

The gateway forwards whatever message list the frontend sends, without truncation. A long enough conversation will eventually hit the upstream model's context window and fail upstream.

Mid-stream upstream errors are in-band

Once SSE headers go out (HTTP 200), the gateway can no longer change the status code. Mid-stream upstream failures are surfaced as `data: [ERROR ...]` followed by `data: [DONE]`; the HTTP status stays 200. The corresponding `upstream_error` ops_event captures the operational signal regardless.

Troubleshooting

401 Unauthorized on every request

`OPENAGENT_API_KEY` in openagent-frontend's environment doesn't match `OPENAGENT_API_KEY` in this repo's `.env`.

502 / "Cannot reach openagent-infra"

openagent-api cannot establish a TCP connection to `OPENAGENT_INFRA_URL`. Check if openagent-infra is running.

`openagent-api` won't start: "OPENAGENT_API_KEY is required"

`.env` is missing or `OPENAGENT_API_KEY` is blank.

WARNING | openagent-logger returned HTTP 401

Every event POST is returning 401 from openagent-logger. `LOGGER_API_KEY` mismatch or `LOGGER_HMAC_SECRET` mismatch.

WARNING | LoggerClient queue full (max=1000); dropped oldest event

The in-memory queue has reached its capacity and is shedding old events to make room for new ones. openagent-logger is down/slow, or there is a massive burst of traffic.

Events aren't showing up in openagent-logger even though `/chat` works

The fire-and-forget contract means /chat works regardless of whether events land. Check openagent-api's logs for **WARNING | LoggerClient POST failed**.

HTTP 422 on /chat with a valid request

Most common cause: invalid **reasoning_effort** value. Pydantic only accepts **low**, **medium**, or **high**.

Port 8001 already in use

Change the left side of **"8001:8001"** in **docker-compose.yml** to a free port.

Healthcheck shows **(unhealthy)** in **docker ps**

The container's HEALTHCHECK probes **/health** with the X-API-Key from **OPENAGENT_API_KEY**. If that env var isn't set, the probe fails.

Reasoning chain shows up as JSON instead of clean text in the chat bubble

The frontend isn't decoding the OpenAI ChatCompletion chunk wrapper. This is a frontend bug, not a gateway bug.

License

Copyright © 2026 William McKeon.

Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at

<http://www.apache.org/licenses/LICENSE-2.0>

Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.

Maintainer

William McKeon (github.com/william-mckeon)